Tend: Hackathon Plan and Antigravity Build Guide

Amazon Developer Hackathon (Build, Ship, Shape) · Primary track: Alexa+ · Mini challenges: AWS Builder and Open Source · Build tool: Gemini 3.8 Flash in Google Antigravity

Read this first: the deadline

Listings give the submission deadline as 23 October 2026, 12:00 PM PT, which is 19:00 UTC today. Confirm the exact time on the Devpost page now. This plan is ordered so a submittable project exists after Phase 6. Phases 7 to 9 take it from demo to near-commercial and can continue after submitting if time runs out.

1. The solution

Track and idea

Primary track: **Alexa+**, with **AWS Builder** and **Open Source** as mini challenges.

- **Why Alexa+:** no hardware is needed and the spec is public. Alexa+ is the MCP client and supports MCP Apps for rendering interactive UI, so the project gets both a voice experience and a visual one.
- **Why not the others:** Bee needs live data from a real wearable. Ring's API surface could not be verified, which is a higher risk for a one-day build.
- **The field is crowded with small utilities.** Public listings show entries for appliance diagnosis, fitness logging, and pantry tracking, and at least one already pairs a simulator with both mini challenges. Winning needs depth, a real human problem, and ecosystem value.

Tend is a family care circle that lives inside Alexa+, built for people looking after an aging parent.

- Mom says, "Alexa, tell Tend I took my morning pills."
- Her daughter asks, "Alexa, how's Mom doing today?" and hears a spoken briefing on meds, check-ins, and appointments.

- A missed dose or skipped check-in escalates to the right sibling by SMS or push. Siblings also see who has claimed which tasks, so nobody calls twice and nobody is forgotten.

Nobody has to install another app.

Design decisions that win points

1. **Be honest about the platform.** Alexa+ is the client, so the server cannot start a conversation. Tend uses pull for voice (briefings on request) and push for alerts (SNS to phones). Saying this explicitly in the demo shows real understanding of the ecosystem.
2. **Pass the latency requirement structurally.** Alexa+ requires a round-trip response under 500 ms. There is no LLM call in the request path. Briefings are deterministic and built from a few DynamoDB reads. Bedrock runs asynchronously on a schedule to write cached "insights" (for example, "morning dose missed 3 of 7 days, worth mentioning to the doctor").
3. **Treat auth as a first-class feature.** Alexa+ expects a bare 401 (no WWW-Authenticate header) on unauthenticated calls, RFC 9728 protected-resource metadata, auth-server metadata advertising S256 PKCE, and the resource parameter on both authorize and token requests. Dynamic Client Registration, Client ID Metadata Documents, OIDC, and step-up authorization are not supported yet. Most hackathon entries will cut corners here. This one will not.
4. **Voice-first tool design.** At most 8 tools, a one-sentence description for each, and every tool returns a short spoken string plus structured data.
5. **Safety scope.** Tend records and coordinates. It never advises on dosing or diagnoses, and it states that it is not for emergencies.
6. **Ecosystem gift (Open Source mini challenge).** `alexa-mcp-check` is a separate Apache-2.0 CLI that tests any MCP server against the Alexa+ checklist and lints tool descriptions for voice use. It runs on Tend in CI, and other builders can use it too.

Tools (v1)

Tool	Used by	Purpose
`log_medication`	Elder	Record taken or skipped for a scheduled dose
`record_check_in`	Elder	"Tell Tend I'm fine" with an optional note

Tool	Used by	Purpose
`get_daily_briefing`	Caregiver	Spoken summary: meds, check-ins, tasks, appointments
`list_care_tasks` / `claim_care_task`	Caregiver	Shared board that avoids duplicate work
`add_care_note`	Both	“Tell the family Dad’s knee is better”
`list_appointments`	Both	What is coming up
`notify_circle`	Caregiver	Escalation, with an explicit confirm step for urgent messages

An MCP App UI (`ui://tend/care-board`) renders the care board on screen devices.

Architecture

```
Alexa+ (MCP client) ── HTTPS, Streamable HTTP ──▶ ALB ▶ Fargate: tend-mcp
(TypeScript)
   |  account linking: OAuth 2.1 + PKCE S256            ├─ POST /mcp  (tools,
resources, MCP App)
   └──────────────────────────────────────────────────▶   ├─ /oauth/*, /.well-
known/*
                                                         └─ DynamoDB single
table (KMS, PITR)
EventBridge Scheduler ▶ Lambda digest worker ▶ Bedrock (async insights,
cached)
                                              └▶ SNS ▶ caregivers (SMS / push /
email)
Tend Console (React on S3 + CloudFront) ── same API ──▶ med schedules,
invites, prefs
```

Fargate rather than Lambda means no cold starts against the 500 ms budget.

Judging map

Criterion	What proves it
Tech Implementation	Spec-compliant server, S256 PKCE with `resource`, p95 latency chart, add-on deployed and tested in the simulator, conformance tool green

Criterion	What proves it
Design	Few voice-friendly tools, spoken-form outputs, confirm-before-escalate, visual care board
Potential Impact	Caregiving is a large, real problem (verify any statistic before putting it in the pitch). Freemium for families, B2B2C for home-care agencies
Quality of Idea	Pull and push architecture, plus an open-source tool that helps other builders

Risks

Risk	Mitigation
Deadline is today	Use the sprint order in section 3
Account linking fails	Run conformance checks in Phase 3, before any Alexa deploy
Web simulator access is slow	Fallback is your own console plus the Local Inspector, recorded for the demo
Flash drifts on security code	Human review of every diff in Phase 3, with tests as the oracle
Claiming things that do not work	Enter a mini challenge only if the video and code evidence it

2. Operating Gemini 3.8 Flash in Antigravity

- **What the model offers:** a 1M-token context, up to 64K output tokens, and low/medium/high thinking levels. It is the default model behind the Antigravity agent and SDK, and it can use more tokens on long, complex tasks by design, so keep tasks small and scoped.
- **Thinking levels:** if your build exposes them, use high for Phases 1 and 3, medium for most work, and low for scaffolding and docs.
- **One phase per agent task.** Ask for the plan artifact first, approve it, then let it run. End every task with the gate command.

- **Parallelize** with the Manager view once the Phase 1 spec is frozen: Agent A takes Phases 2 and 4, Agent B takes Phase 3, Agent C takes Phase 8.
- **Rules:** Antigravity reads AGENTS.md at the repo root, and GEMINI.md overrides it. Rules and workflows are capped at 12,000 characters per file, and skills live in SKILL.md files.
- **Folder ambiguity:** `.agents/` versus `.agent/` has been inconsistent across tools and docs (newer installs write `.agents/`, with `.agent/` as a legacy fallback). Use whichever path your Antigravity panel creates, and do not gitignore it.

Starter AGENTS.md

```
# AGENTS.md — Tend (Alexa+ MCP add-on)
## Mission
Voice-first family care coordination, exposed as an MCP server for Alexa+.
## Stack
TypeScript/Node 22, pnpm workspaces (apps/mcp-server, apps/console,
packages/core,
packages/conformance, infra/cdk), Hono, official @modelcontextprotocol/sdk
(version
supporting spec 2025-11-25), Zod, Vitest, AWS CDK v2, DynamoDB single-table,
Bedrock.
## Commands
pnpm typecheck | lint | test | test:int | verify (all + conformance +
latency)
## Alexa+ hard requirements
- Streamable HTTP, stateless. Unauthenticated => 401 with NO WWW-Authenticate
header.
- RFC 9728 PRM; AS metadata at /.well-known/oauth-authorization-server with
S256.
- OAuth 2.1 code + PKCE S256; `resource` on authorize AND token; Bearer in
header only.
- p95 < 250 ms locally. No LLM calls or network fan-out in the request path.
## Tool design
<=8 tools, verb_noun names, one-sentence descriptions. Every tool returns
{speech (<=280 chars, plain words, no markdown), data}. Risky actions need
confirm=true.
## Safety
Never give medical advice or dosing guidance. Record and coordinate only.
## Security
No secrets in repo (.env.example only). Zod-validate all input. Authorize
every tool
```

```
call against circleId from the token. Audit-log every write.
## Process
Plan artifact first. Small commits. Tests ship with code. Do not refactor
unrelated files.
When docs or tools misbehave, append to FRICTION.md (task, steps, expected,
actual,
severity, workaround, suggestion).
```

3. Phase-by-phase plan

Sprint order if you are on today's clock

Block	Work
0:00–0:45	Phases 0 and 1 (spec-lite)
0:45–3:00	Phases 2 and 4 combined (in-memory first, DynamoDB if time allows)
3:00–4:30	Phase 3 (auth)
4:30–6:00	Phase 6 (cloudflared tunnel, `alexa-ai deploy`, simulator)
6:00–8:00	Record the video and complete Phase 10

Do Phase 8 as a minimal version in parallel. Skip Phase 7. Do Phase 5 only for the parts you can show working (Bedrock insight, SNS alert), and do not claim AWS Builder otherwise.

Phase 0: Foundations (30 min, low thinking)

```
Create the pnpm monorepo described in AGENTS.md with typed empty packages,
GitHub
Actions CI (typecheck, lint, test), .env.example, FRICTION.md, README
skeleton.
Licenses: main repo proprietary placeholder; packages/conformance Apache-2.0.
Fetch https://developer.amazon.com/docs/llms.txt, then create a skill
alexa-mcp-compliance/SKILL.md that turns the MCP QuickStart, design guide,
and
certification requirements into a checklist with source URLs. No feature
code.
```

Gate: `pnpm verify` passes on the empty skeleton.

Phase 1: Spec and threat model (45 min, high thinking, no code)

```
Plan only. Write docs/SPEC.md: 3 personas, 6 voice journeys with sample
utterances,
JSON Schema for every tool plus example speech outputs, DynamoDB single-table
design
with access patterns, OAuth sequence diagrams (Mermaid), STRIDE threat table,
error
taxonomy, and a latency budget. List up to 5 open questions instead of
guessing.
```

Gate: you review it and tag `spec-v1`.

Phase 2: MCP core (medium)

```
Implement apps/mcp-server per docs/SPEC.md: Streamable HTTP endpoint,
protocol-version
negotiation, tools/list, tools/call, all 8 tools backed by an in-memory
repository
interface, Zod validation, structured logs with request IDs. Add integration
tests
using the MCP SDK client. No auth yet.
```

Gate: integration tests pass and the MCP Inspector can list and call tools.

Phase 3: OAuth 2.1 and discovery (high, review every diff yourself)

```
Implement the authorization server and resource protection: authorization
endpoint
with PKCE S256 only, token endpoint with `resource` validation, refresh
tokens
(rotation), PRM per RFC 9728, AS metadata. Unauthenticated /mcp returns 401
with no
WWW-Authenticate header. Bearer in header only (reject query-string tokens).
Bind
tokens to the resource and circle. Write negative tests: missing verifier,
wrong
resource, replayed code, expired token.
```

Gate: all negative tests pass, and the conformance CLI (even in rough form) passes the auth checks.

Phase 4: Persistence and domain logic (medium)

```
Replace the in-memory repo with DynamoDB single-table (use DynamoDB Local for
tests).
Implement circles, members with roles, medication schedules, dose events,
check-ins,
tasks, notes, appointments, an append-only audit log, and idempotency keys on
writes.
Briefing = deterministic template over <=4 queries. Add a p95 latency test.
```

Gate: p95 under 250 ms locally across 100 calls.

Phase 5: AWS deployment and async intelligence (medium)

```
Write infra/cdk: VPC, Fargate service behind ALB with ACM cert, DynamoDB
(KMS, PITR),
Secrets Manager, EventBridge Scheduler -> Lambda digest worker -> Bedrock
(Converse
API) writing cached insights, SNS escalation, CloudWatch dashboards and
alarms (p95,
5xx, auth failures). Create docs/AWS_BUILDER.md listing each AWS service and
exactly
how it is used. Use a Bedrock model available in the deploy region.
```

Gate: `cdk deploy` succeeds, `/mcp` answers over HTTPS, and a scheduled digest writes an insight.

Phase 6: Alexa+ add-on and simulator (medium)

```
Using the Alexa AI CLI (alexa-ai configure / new mcp / deploy), create
addon-package/addon.json: name <=30 chars, shortDescription <=123,
fullDescription
<=4000, 3-4 example phrases, privacy and terms URLs (host simple HTTPS
pages), 6 light
icon sizes (72, 64, 88, 126, 180, 241), one 600x900 carousel image. Add the
MCP App
care-board UI. Deploy to the development stage and write
docs/SIMULATOR_TEST.md with
the utterances tested and the results.
```

Gate: all journeys work in the web simulator. Alexa+ refreshes tool information only on deploy, so redeploy after every tool-description change. Tune the descriptions until Alexa+ picks the right tool reliably.

Phase 7: Tend Console (can run in parallel, low to medium)

```
Build apps/console (Vite + React): sign-in, create circle, invite by code,
medication
schedule editor, notification preferences, data export and delete-my-account.
Add a
seed script and a scripted demo scenario.
```

Setting up medications by voice is a poor experience, so this web UI is where schedules should be created.

Phase 8: alexa-mcp-check (parallel agent, medium)

```
Build packages/conformance as a CLI: npx alexa-mcp-check <url>. Checks:
Streamable
HTTP, bare 401, PRM, AS metadata with S256, resource handling, version
negotiation,
p95 latency over N calls vs 500 ms, and a tool-description linter (length,
ambiguity,
parameter count, speech-friendliness). Output a human report plus JSON. Add
README,
Apache-2.0 LICENSE, CI, and tests against fixtures.
```

Gate: it passes against Tend and fails against a deliberately broken fixture. Publish it to a public repo and npm. A fork or PR to a public MCP or docs repo can serve as a second contribution if time allows.

Phase 9: Commercial hardening (medium)

```
Run a security review against the docs/SPEC.md threat table and fix gaps. Add
rate
limits, request size limits, input fuzz tests, dependency audit, and SBOM.
Write:
privacy policy, terms, a not-a-medical-device and not-for-emergencies notice,
data
retention and deletion procedure, runbook, backup/restore test, load test at
50 rps,
and a cost-per-circle estimate. Pre-check against the Alexa+ Functional and
Policy
Requirements pages and list any gaps in docs/CERTIFICATION_PRECHECK.md.
```

Have someone qualified review the privacy and compliance wording before real users touch the product.

Phase 10: Submission pack (low thinking, but do not rush it)

- **Repo access:** if the main repo is private, add `knmeiss`, `giolaq`, `anishamalde`, `mosesroth`, and `emersonsklar` as collaborators, and share with `testing@devpost.com` too. Each person must accept the invitation, and invitations expire after 7 days, so add them as you submit and re-send any that lapsed. The open-source tool lives in its own public repo with its license.
- **Required for the track:** the code must visibly use the MCP SDK (the import, the server entry point, and the loaded config). Point to those files in the README.
- **Demo video, under 3 minutes, public on YouTube or Vimeo, in English:**
  - 0:00–0:15: the problem
  - 0:15–1:45: live in the Alexa+ simulator (log a dose, ask for a briefing, trigger an escalation, show the care board)
  - 1:45–2:15: architecture, AWS services, and a latency chart
  - 2:15–2:40: `npx alexa-mcp-check` going green
  - 2:40–3:00: impact and roadmap
  - Use no third-party music or footage.

- **Written fields:** project description, tracks and mini challenges entered, and a note that the project was built during the window (only if true). The Open Source entry also needs the contribution URL, repo URL, GitHub username, and a short description.
- **Product feedback:** for every tool you used, cover what you used it for, what worked, what needs work, how onboarding felt, and whether you would build with it again. Describe the AWS services here too.
- **Friction log and feature requests:** a friction log can earn up to a 10% judging bonus. Use the entries your agent has been writing to FRICTION.md since Phase 0.

Roadmap slide: Ring door and motion events as signal sources, and a Fire TV care-board card. Present these as future work, not as built.

Sources

- Hackathon page: https://amazonappdev2026.devpost.com/ (submission deadline per third-party listings, e.g. https://internshala.com/competitions/?p=11209)
- Alexa+ MCP QuickStart: https://developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart.html
- Alexa+ MCP Toolkit Overview: https://developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-overview.html
- Gemini 3.8 Flash: https://ai.google.dev/gemini-api/docs/latest-model
- Antigravity rules and AGENTS.md: https://docs.coingecko.com/docs/ai-agent-hub/antigravity and https://thepromptshelf.dev/blog/google-antigravity-agents-md-rules-guide-2026